

Wisconsin-Breast-Cancer-Diagnosis-Project.R

mengkiat

2025-07-12

```
#Wisconsin Breast Cancer Diagnosis Project
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(data.table)
```

```
library(caTools)
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:data.table':
```

```
##
```

```
##      between, first, last
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```
library(randomForest)
```

```
## randomForest 4.7-1.1
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'

## The following object is masked from 'package:dplyr':
##
##      combine

## The following object is masked from 'package:ggplot2':
##
##      margin
```

```
library(rpart)
library(rpart.plot)

setwd("/Users/mengkiat/Library/CloudStorage/OneDrive-NanyangTechnologicalUniversity/Class/ARCHIVE/Y2SEM1")

data.dt <- fread("wisconsin breast cancer data.csv")

head(data.dt)
```

```
##      id diagnosis radius_mean texture_mean perimeter_mean area_mean
##      <int>      <char>      <num>      <num>      <num>      <num>
## 1:   842302         M       17.99       10.38       122.80      1001.0
## 2:   842517         M       20.57       17.77       132.90      1326.0
## 3:  84300903         M       19.69       21.25       130.00      1203.0
## 4:  84348301         M       11.42       20.38       77.58       386.1
## 5:  84358402         M       20.29       14.34       135.10      1297.0
## 6:   843786         M       12.45       15.70       82.57       477.1
##      smoothness_mean compactness_mean concavity_mean concave points_mean
##      <num>      <num>      <num>      <num>      <num>
## 1:      0.11840      0.27760      0.3001      0.14710
## 2:      0.08474      0.07864      0.0869      0.07017
## 3:      0.10960      0.15990      0.1974      0.12790
## 4:      0.14250      0.28390      0.2414      0.10520
## 5:      0.10030      0.13280      0.1980      0.10430
## 6:      0.12780      0.17000      0.1578      0.08089
##      symmetry_mean fractal_dimension_mean radius_se texture_se perimeter_se
##      <num>      <num>      <num>      <num>      <num>
## 1:      0.2419      0.07871      1.0950      0.9053      8.589
## 2:      0.1812      0.05667      0.5435      0.7339      3.398
## 3:      0.2069      0.05999      0.7456      0.7869      4.585
## 4:      0.2597      0.09744      0.4956      1.1560      3.445
## 5:      0.1809      0.05883      0.7572      0.7813      5.438
## 6:      0.2087      0.07613      0.3345      0.8902      2.217
##      area_se smoothness_se compactness_se concavity_se concave points_se
##      <num>      <num>      <num>      <num>      <num>
## 1:  153.40      0.006399      0.04904      0.05373      0.01587
## 2:   74.08      0.005225      0.01308      0.01860      0.01340
## 3:   94.03      0.006150      0.04006      0.03832      0.02058
## 4:   27.23      0.009110      0.07458      0.05661      0.01867
## 5:   94.44      0.011490      0.02461      0.05688      0.01885
## 6:   27.19      0.007510      0.03345      0.03672      0.01137
##      symmetry_se fractal_dimension_se radius_worst texture_worst perimeter_worst
```

```
##          <num>          <num>          <num>          <num>          <num>
## 1:      0.03003      0.006193      25.38      17.33      184.60
## 2:      0.01389      0.003532      24.99      23.41      158.80
## 3:      0.02250      0.004571      23.57      25.53      152.50
## 4:      0.05963      0.009208      14.91      26.50      98.87
## 5:      0.01756      0.005115      22.54      16.67      152.20
## 6:      0.02165      0.005082      15.47      23.75      103.40
##      area_worst smoothness_worst compactness_worst concavity_worst
##          <num>          <num>          <num>          <num>
## 1:      2019.0      0.1622      0.6656      0.7119
## 2:      1956.0      0.1238      0.1866      0.2416
## 3:      1709.0      0.1444      0.4245      0.4504
## 4:       567.7      0.2098      0.8663      0.6869
## 5:      1575.0      0.1374      0.2050      0.4000
## 6:       741.6      0.1791      0.5249      0.5355
##      concave points_worst symmetry_worst fractal_dimension_worst
##          <num>          <num>          <num>
## 1:      0.2654      0.4601      0.11890
## 2:      0.1860      0.2750      0.08902
## 3:      0.2430      0.3613      0.08758
## 4:      0.2575      0.6638      0.17300
## 5:      0.1625      0.2364      0.07678
## 6:      0.1741      0.3985      0.12440
```

```
str(data.dt)
```

```
## Classes 'data.table' and 'data.frame': 569 obs. of 32 variables:
```

```
## $ id          : int  842302 842517 84300903 84348301 84358402 843786 844359 84458202 844
## $ diagnosis   : chr  "M" "M" "M" "M" ...
## $ radius_mean : num  18 20.6 19.7 11.4 20.3 ...
## $ texture_mean : num  10.4 17.8 21.2 20.4 14.3 ...
## $ perimeter_mean : num  122.8 132.9 130 77.6 135.1 ...
## $ area_mean    : num  1001 1326 1203 386 1297 ...
## $ smoothness_mean : num  0.1184 0.0847 0.1096 0.1425 0.1003 ...
## $ compactness_mean : num  0.2776 0.0786 0.1599 0.2839 0.1328 ...
## $ concavity_mean : num  0.3001 0.0869 0.1974 0.2414 0.198 ...
## $ concave points_mean : num  0.1471 0.0702 0.1279 0.1052 0.1043 ...
## $ symmetry_mean  : num  0.242 0.181 0.207 0.26 0.181 ...
## $ fractal_dimension_mean : num  0.0787 0.0567 0.06 0.0974 0.0588 ...
## $ radius_se      : num  1.095 0.543 0.746 0.496 0.757 ...
## $ texture_se     : num  0.905 0.734 0.787 1.156 0.781 ...
## $ perimeter_se   : num  8.59 3.4 4.58 3.44 5.44 ...
## $ area_se        : num  153.4 74.1 94 27.2 94.4 ...
## $ smoothness_se  : num  0.0064 0.00522 0.00615 0.00911 0.01149 ...
## $ compactness_se : num  0.049 0.0131 0.0401 0.0746 0.0246 ...
## $ concavity_se   : num  0.0537 0.0186 0.0383 0.0566 0.0569 ...
## $ concave points_se : num  0.0159 0.0134 0.0206 0.0187 0.0188 ...
## $ symmetry_se    : num  0.03 0.0139 0.0225 0.0596 0.0176 ...
## $ fractal_dimension_se : num  0.00619 0.00353 0.00457 0.00921 0.00511 ...
## $ radius_worst   : num  25.4 25 23.6 14.9 22.5 ...
## $ texture_worst  : num  17.3 23.4 25.5 26.5 16.7 ...
## $ perimeter_worst : num  184.6 158.8 152.5 98.9 152.2 ...
## $ area_worst     : num  2019 1956 1709 568 1575 ...
## $ smoothness_worst : num  0.162 0.124 0.144 0.21 0.137 ...
```

```
## $ compactness_worst      : num  0.666 0.187 0.424 0.866 0.205 ...
## $ concavity_worst       : num  0.712 0.242 0.45 0.687 0.4 ...
## $ concave points_worst  : num  0.265 0.186 0.243 0.258 0.163 ...
## $ symmetry_worst        : num  0.46 0.275 0.361 0.664 0.236 ...
## $ fractal_dimension_worst: num  0.1189 0.089 0.0876 0.173 0.0768 ...
## - attr(*, ".internal.selfref")=<externalptr>
```

```
#data cleaning
```

```
#Checking for null values
```

```
sum(is.na(data.dt))
```

```
## [1] 0
```

```
#0
```

```
#Omitting redundant 'id' variable
```

```
data.dt <- data.dt[,-1]
```

```
head(data.dt)
```

```
##      diagnosis radius_mean texture_mean perimeter_mean area_mean smoothness_mean
##      <char>      <num>      <num>      <num>      <num>      <num>
## 1:      M      17.99      10.38      122.80      1001.0      0.11840
## 2:      M      20.57      17.77      132.90      1326.0      0.08474
## 3:      M      19.69      21.25      130.00      1203.0      0.10960
## 4:      M      11.42      20.38      77.58      386.1      0.14250
## 5:      M      20.29      14.34      135.10      1297.0      0.10030
## 6:      M      12.45      15.70      82.57      477.1      0.12780
##      compactness_mean concavity_mean concave points_mean symmetry_mean
##      <num>      <num>      <num>      <num>
## 1:      0.27760      0.3001      0.14710      0.2419
## 2:      0.07864      0.0869      0.07017      0.1812
## 3:      0.15990      0.1974      0.12790      0.2069
## 4:      0.28390      0.2414      0.10520      0.2597
## 5:      0.13280      0.1980      0.10430      0.1809
## 6:      0.17000      0.1578      0.08089      0.2087
##      fractal_dimension_mean radius_se texture_se perimeter_se area_se
##      <num>      <num>      <num>      <num>      <num>
## 1:      0.07871      1.0950      0.9053      8.589      153.40
## 2:      0.05667      0.5435      0.7339      3.398      74.08
## 3:      0.05999      0.7456      0.7869      4.585      94.03
## 4:      0.09744      0.4956      1.1560      3.445      27.23
## 5:      0.05883      0.7572      0.7813      5.438      94.44
## 6:      0.07613      0.3345      0.8902      2.217      27.19
##      smoothness_se compactness_se concavity_se concave points_se symmetry_se
##      <num>      <num>      <num>      <num>      <num>
## 1:      0.006399      0.04904      0.05373      0.01587      0.03003
## 2:      0.005225      0.01308      0.01860      0.01340      0.01389
## 3:      0.006150      0.04006      0.03832      0.02058      0.02250
## 4:      0.009110      0.07458      0.05661      0.01867      0.05963
## 5:      0.011490      0.02461      0.05688      0.01885      0.01756
## 6:      0.007510      0.03345      0.03672      0.01137      0.02165
```

```
##      fractal_dimension_se radius_worst texture_worst perimeter_worst area_worst
##      <num>           <num>           <num>           <num>           <num>
## 1:      0.006193      25.38      17.33      184.60      2019.0
## 2:      0.003532      24.99      23.41      158.80      1956.0
## 3:      0.004571      23.57      25.53      152.50      1709.0
## 4:      0.009208      14.91      26.50      98.87      567.7
## 5:      0.005115      22.54      16.67      152.20      1575.0
## 6:      0.005082      15.47      23.75      103.40      741.6
##      smoothness_worst compactness_worst concavity_worst concave points_worst
##      <num>           <num>           <num>           <num>
## 1:      0.1622      0.6656      0.7119      0.2654
## 2:      0.1238      0.1866      0.2416      0.1860
## 3:      0.1444      0.4245      0.4504      0.2430
## 4:      0.2098      0.8663      0.6869      0.2575
## 5:      0.1374      0.2050      0.4000      0.1625
## 6:      0.1791      0.5249      0.5355      0.1741
##      symmetry_worst fractal_dimension_worst
##      <num>           <num>
## 1:      0.4601      0.11890
## 2:      0.2750      0.08902
## 3:      0.3613      0.08758
## 4:      0.6638      0.17300
## 5:      0.2364      0.07678
## 6:      0.3985      0.12440
```

```
#Converting Diagnosis column B(benign), malignant(M) to 0,1
```

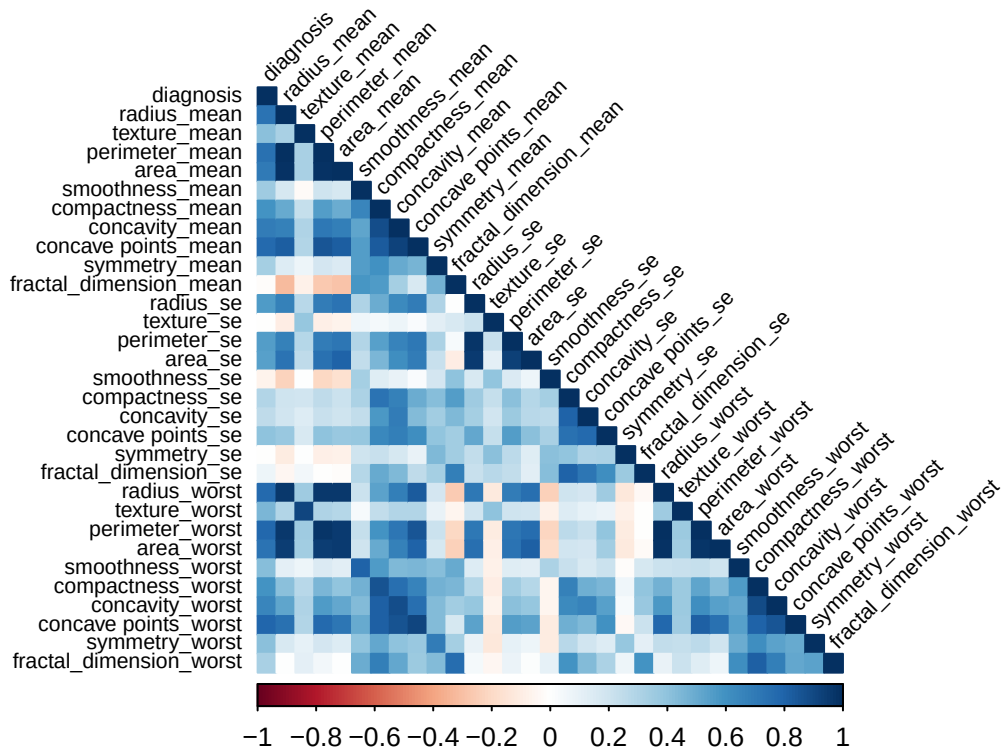
```
data.dt$diagnosis <- ifelse(data.dt$diagnosis=="B",0,1)
```

```
#Initial variable selection
```

```
corr1 <- cor(data.dt)
```

```
#Visual representation
```

```
corrplot(corr1,type = 'lower', method = 'color', tl.col = 'black', tl.srt = 45, tl.cex = 0.7)
```



#Identifying pairs of variables with correlation coefficient >0.8

```
pairs <- findCorrelation(corr1, cutoff = 0.8)
colnames(corr1)[pairs]
```

```
## [1] "concavity_mean"      "concave points_mean" "concave points_worst"
## [4] "compactness_mean"    "concavity_worst"     "perimeter_worst"
## [7] "radius_worst"        "perimeter_mean"      "area_worst"
## [10] "compactness_worst"   "radius_mean"         "perimeter_se"
## [13] "area_se"             "compactness_se"      "smoothness_mean"
## [16] "texture_mean"
```

#Dropping one variable from each pair

```
to_drop <- c("concavity_mean", "compactness_mean", "concavity_worst", "radius_worst", "compactness_worst", "area_worst", "perimeter_worst", "smoothness_worst", "concavity_se", "compactness_se", "concave points_worst", "symmetry_worst", "fractal_dimension_worst")
```

```
initial_data <- select(data.dt, -to_drop)
```

```
## Warning: Using an external vector in selections was deprecated in tidysselect 1.1.0.
```

```
## i Please use 'all_of()' or 'any_of()' instead.
```

```
## # Was:
```

```
## data %>% select(to_drop)
```

```
##
```

```
## # Now:
```

```
## data %>% select(all_of(to_drop))
```

```
##
```

```
## See <https://tidysselect.r-lib.org/reference/faq-external-vector.html>.
```

```
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
str(initial_data)
```

```
## Classes 'data.table' and 'data.frame':  569 obs. of  23 variables:
## $ diagnosis      : num  1 1 1 1 1 1 1 1 1 1 ...
## $ texture_mean   : num  10.4 17.8 21.2 20.4 14.3 ...
## $ perimeter_mean : num  122.8 132.9 130 77.6 135.1 ...
## $ area_mean      : num  1001 1326 1203 386 1297 ...
## $ concave_points_mean : num  0.1471 0.0702 0.1279 0.1052 0.1043 ...
## $ symmetry_mean   : num  0.242 0.181 0.207 0.26 0.181 ...
## $ fractal_dimension_mean : num  0.0787 0.0567 0.06 0.0974 0.0588 ...
## $ radius_se       : num  1.095 0.543 0.746 0.496 0.757 ...
## $ texture_se       : num  0.905 0.734 0.787 1.156 0.781 ...
## $ perimeter_se     : num  8.59 3.4 4.58 3.44 5.44 ...
## $ area_se          : num  153.4 74.1 94 27.2 94.4 ...
## $ smoothness_se    : num  0.0064 0.00522 0.00615 0.00911 0.01149 ...
## $ concavity_se     : num  0.0537 0.0186 0.0383 0.0566 0.0569 ...
## $ concave_points_se : num  0.0159 0.0134 0.0206 0.0187 0.0188 ...
## $ symmetry_se       : num  0.03 0.0139 0.0225 0.0596 0.0176 ...
## $ fractal_dimension_se : num  0.00619 0.00353 0.00457 0.00921 0.00511 ...
## $ texture_worst     : num  17.3 23.4 25.5 26.5 16.7 ...
## $ perimeter_worst   : num  184.6 158.8 152.5 98.9 152.2 ...
## $ area_worst        : num  2019 1956 1709 568 1575 ...
## $ smoothness_worst  : num  0.162 0.124 0.144 0.21 0.137 ...
## $ concave_points_worst : num  0.265 0.186 0.243 0.258 0.163 ...
## $ symmetry_worst    : num  0.46 0.275 0.361 0.664 0.236 ...
## $ fractal_dimension_worst: num  0.1189 0.089 0.0876 0.173 0.0768 ...
## - attr(*, ".internal.selfref")=<externalptr>
```

```
ncol(initial_data)
```

```
## [1] 23
```

```
#Factorise diagnosis
initial_data$diagnosis <- factor(initial_data$diagnosis)
class(initial_data$diagnosis)
```

```
## [1] "factor"
```

```
#Identifying important variables
```

```
#Log Regression
logmodel1 <- glm(diagnosis~., data = initial_data, family = binomial())
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```

vim1 <- as.data.frame(varImp(logmodel1, scale=FALSE))
vim1 <- data.frame(overall = vim1$Overall,
                  names = rownames(vim1))
vim1 <- vim1[order(vim1$overall,decreasing = T),]
vim1

```

```

##      overall              names
## 4  2.4964355  'concave points_mean'
## 15 1.9484144   fractal_dimension_se
## 22 1.6532891 fractal_dimension_worst
## 16 1.6021534      texture_worst
## 21 1.3933995      symmetry_worst
## 13 1.3407730  'concave points_se'
## 2  1.3187122      perimeter_mean
## 14 1.3186723      symmetry_se
## 10 1.2119338        area_se
## 9  1.1111234      perimeter_se
## 3  0.8604253        area_mean
## 17 0.8233938      perimeter_worst
## 6  0.7702434 fractal_dimension_mean
## 8  0.6604628      texture_se
## 11 0.5149552      smoothness_se
## 18 0.5099759        area_worst
## 1  0.4612832      texture_mean
## 19 0.4003602      smoothness_worst
## 5  0.3165469      symmetry_mean
## 7  0.2900739        radius_se
## 12 0.1871269      concavity_se
## 20 0.1237878  'concave points_worst'

```

```

write.csv(vim1,file="vim1.csv",row.names = FALSE)

```

```

#RandomForest

```

```

set.seed(100)

```

```

colnames(initial_data)

```

```

## [1] "diagnosis"          "texture_mean"
## [3] "perimeter_mean"     "area_mean"
## [5] "concave points_mean" "symmetry_mean"
## [7] "fractal_dimension_mean" "radius_se"
## [9] "texture_se"         "perimeter_se"
## [11] "area_se"           "smoothness_se"
## [13] "concavity_se"       "concave points_se"
## [15] "symmetry_se"        "fractal_dimension_se"
## [17] "texture_worst"      "perimeter_worst"
## [19] "area_worst"         "smoothness_worst"
## [21] "concave points_worst" "symmetry_worst"
## [23] "fractal_dimension_worst"

```

```

names(initial_data) <-make.names(names(initial_data))

```



```

rf_model1 <- randomForest(diagnosis~.,data=initial_data)
vim2 <- as.data.frame(varImp(rf_model1, scale=FALSE))
vim2 <- data.frame(overall = vim2$Overall,
                  names   = rownames(vim2))
vim2 <- vim2[order(vim2$overall,decreasing = T),]
write.csv(vim2,file="vim2.csv",row.names = FALSE)

#Identify important variables in excel
#Remove unimportant variables
col_to_drop <- c("fractal_dimension_se", "concave.points_se", "concave.points_worst","symmetry_se", "te
f_data<- select(initial_data,-col_to_drop)

```

```

## Warning: Using an external vector in selections was deprecated in tidyselect 1.1.0.
## i Please use 'all_of()' or 'any_of()' instead.
## # Was:
## data %>% select(col_to_drop)
##
## # Now:
## data %>% select(all_of(col_to_drop))
##
## See <https://tidyselect.r-lib.org/reference/faq-external-vector.html>.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```

```
str(f_data)
```

```

## Classes 'data.table' and 'data.frame':  569 obs. of  11 variables:
## $ diagnosis      : Factor w/ 2 levels "0","1": 2 2 2 2 2 2 2 2 2 2 ...
## $ perimeter_mean : num  122.8 132.9 130 77.6 135.1 ...
## $ area_mean      : num  1001 1326 1203 386 1297 ...
## $ concave.points_mean : num  0.1471 0.0702 0.1279 0.1052 0.1043 ...
## $ perimeter_se    : num  8.59 3.4 4.58 3.44 5.44 ...
## $ area_se         : num  153.4 74.1 94 27.2 94.4 ...
## $ texture_worst   : num  17.3 23.4 25.5 26.5 16.7 ...
## $ perimeter_worst : num  184.6 158.8 152.5 98.9 152.2 ...
## $ area_worst      : num  2019 1956 1709 568 1575 ...
## $ symmetry_worst  : num  0.46 0.275 0.361 0.664 0.236 ...
## $ fractal_dimension_worst: num  0.1189 0.089 0.0876 0.173 0.0768 ...
## - attr(*, ".internal.selfref")=<externalptr>

```

```
#Exploring parameters for RF
```

```
#Preparing trainset and testset
```

```
set.seed(100)
```

```

train <- sample.split(Y=f_data$diagnosis,SplitRatio=0.7)
trainset <-subset(f_data,train==T)
testset <-subset(f_data,train==F)

```

```
#Function
```

```
rf_parameter_test <- function(mtry, ntree) {
```

```

  rf_model <- randomForest(diagnosis ~ ., data = trainset, mtry = mtry, ntree = ntree)
  return(rf_model)
}

# Define a grid of mtry and ntree values
RSF <- seq(1, ncol(trainset) - 1, by = 1)
B <- c(25, 100, 500)

RSF

```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
B
```

```
## [1] 25 100 500
```

```

# Train and evaluate the models
results <- list()
for (mtry in RSF) {
  for (ntree in B) {
    rf_model <- rf_parameter_test(mtry, ntree)
    test_predict <- predict(rf_model, newdata = testset)
    accuracy <- sum(test_predict == testset$diagnosis) / nrow(testset)
    results[[paste(mtry, ntree, sep = "_")] <- accuracy
  }
}
results

```

```

## $'1_25'
## [1] 0.9590643
##
## $'1_100'
## [1] 0.9590643
##
## $'1_500'
## [1] 0.9766082
##
## $'2_25'
## [1] 0.9649123
##
## $'2_100'
## [1] 0.9766082
##
## $'2_500'
## [1] 0.9649123
##
## $'3_25'
## [1] 0.9649123
##
## $'3_100'
## [1] 0.9707602
##

```

```
## $'3_500'  
## [1] 0.9707602  
##  
## $'4_25'  
## [1] 0.9824561  
##  
## $'4_100'  
## [1] 0.9766082  
##  
## $'4_500'  
## [1] 0.9766082  
##  
## $'5_25'  
## [1] 0.9707602  
##  
## $'5_100'  
## [1] 0.9824561  
##  
## $'5_500'  
## [1] 0.9824561  
##  
## $'6_25'  
## [1] 0.9766082  
##  
## $'6_100'  
## [1] 0.9824561  
##  
## $'6_500'  
## [1] 0.9824561  
##  
## $'7_25'  
## [1] 0.9649123  
##  
## $'7_100'  
## [1] 0.9766082  
##  
## $'7_500'  
## [1] 0.9824561  
##  
## $'8_25'  
## [1] 0.9883041  
##  
## $'8_100'  
## [1] 0.9824561  
##  
## $'8_500'  
## [1] 0.9824561  
##  
## $'9_25'  
## [1] 0.9649123  
##  
## $'9_100'  
## [1] 0.9824561  
##
```

```
## $'9_500'
## [1] 0.9824561
##
## $'10_25'
## [1] 0.9707602
##
## $'10_100'
## [1] 0.9766082
##
## $'10_500'
## [1] 0.9824561
```

```
table1 <- as.data.frame(results)
table1
```

```
##      X1_25  X1_100  X1_500  X2_25  X2_100  X2_500  X3_25
## 1 0.9590643 0.9590643 0.9766082 0.9649123 0.9766082 0.9649123 0.9649123
##      X3_100  X3_500  X4_25  X4_100  X4_500  X5_25  X5_100
## 1 0.9707602 0.9707602 0.9824561 0.9766082 0.9766082 0.9707602 0.9824561
##      X5_500  X6_25  X6_100  X6_500  X7_25  X7_100  X7_500
## 1 0.9824561 0.9766082 0.9824561 0.9824561 0.9649123 0.9766082 0.9824561
##      X8_25  X8_100  X8_500  X9_25  X9_100  X9_500  X10_25
## 1 0.9883041 0.9824561 0.9824561 0.9649123 0.9824561 0.9824561 0.9707602
##      X10_100  X10_500
## 1 0.9766082 0.9824561
```

```
View(table1)
```

```
#Trying Different seed
set.seed(1000)
```

```
train <- sample.split(Y=f_data$diagnosis,SplitRatio=0.7)
trainset <-subset(f_data,train==T)
testset <-subset(f_data,train==F)

# Train and evaluate the models
results <- list()
for (mtry in RSF) {
  for (ntree in B) {
    rf_model <- rf_parameter_test(mtry, ntree)
    test_predict <- predict(rf_model, newdata = testset)
    accuracy <- sum(test_predict == testset$diagnosis) / nrow(testset)
    results[[paste(mtry, ntree, sep = "_")] <- accuracy
  }
}
```

```
table2 <- as.data.frame(results)
table2
```

```
##      X1_25  X1_100  X1_500  X2_25  X2_100  X2_500  X3_25
## 1 0.9649123 0.9473684 0.9649123 0.9707602 0.9590643 0.9649123 0.9532164
##      X3_100  X3_500  X4_25  X4_100  X4_500  X5_25  X5_100
```

```
## 1 0.9649123 0.9707602 0.9649123 0.9473684 0.9532164 0.9473684 0.9532164
##      X5_500      X6_25      X6_100      X6_500      X7_25      X7_100      X7_500
## 1 0.9590643 0.9707602 0.9415205 0.9532164 0.9649123 0.9532164 0.9532164
##      X8_25      X8_100      X8_500      X9_25      X9_100      X9_500      X10_25
## 1 0.9532164 0.9473684 0.9532164 0.9532164 0.9590643 0.9532164 0.9532164
##      X10_100      X10_500
## 1 0.9532164 0.9590643
```

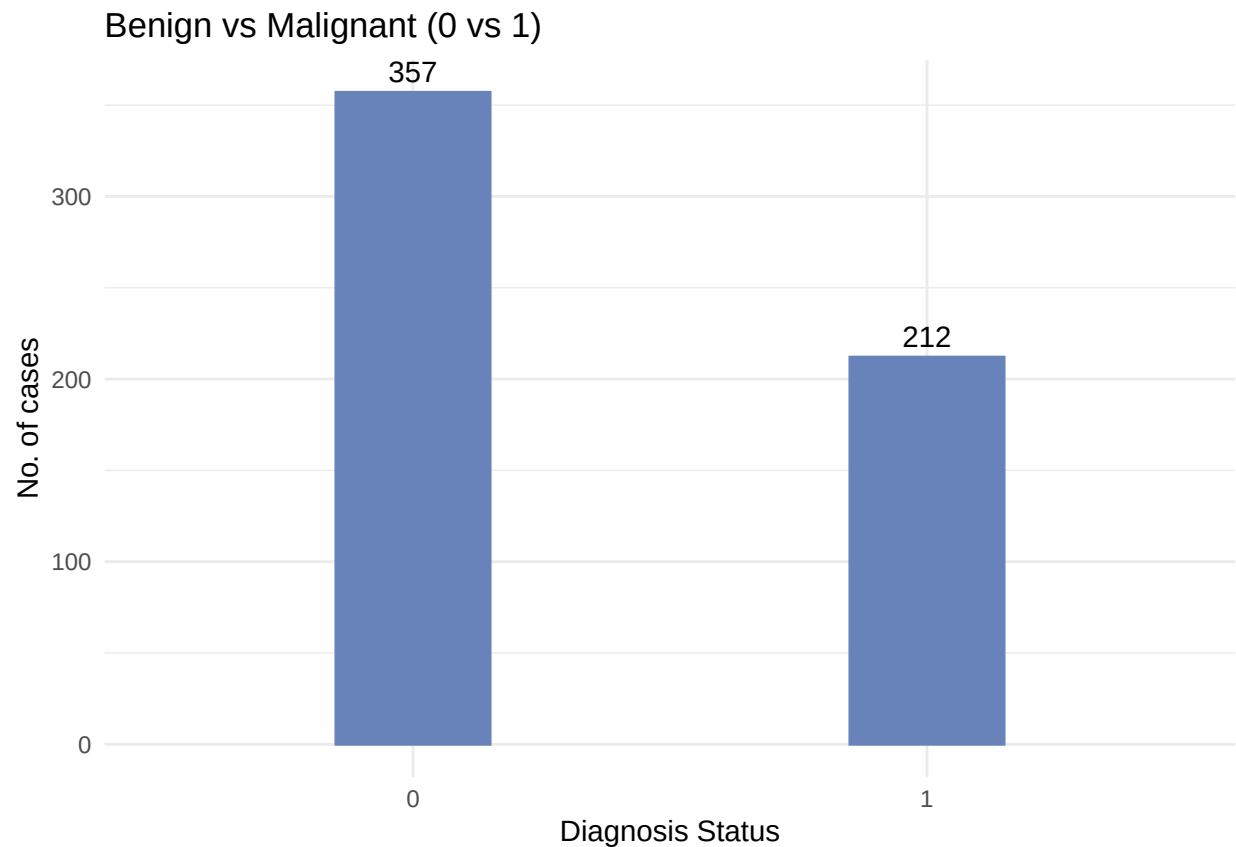
```
View(table2)
```

```
#export in csv
write.csv(table1,file="best_parameter1.csv",row.names = FALSE)
write.csv(table2,file="best_parameter2.csv",row.names = FALSE)

#arranged in excel
#Qn 5
table(f_data$diagnosis)
```

```
##
##      0      1
## 357 212
```

```
ggplot(f_data, aes(x = diagnosis)) +
  geom_bar(width = 0.3, color = '#6883ba',fill='#6883ba') +
  labs(title = "Benign vs Malignant (0 vs 1)",
       x = "Diagnosis Status",
       y = "No. of cases") +
  geom_text(stat = "count", aes(label = after_stat(count)),
           vjust = -0.5, # Adjust vertical position (e.g., -0.5 moves it slightly above the bar)
           color = "black", # Color of the count text
           size = 4) +
  theme_minimal() # A clean theme for the plot
```



```
set.seed(100)
train <- sample.split(Y=f_data$diagnosis,SplitRatio=0.7)
trainset <-subset(f_data,train==T)
testset <-subset(f_data,train==F)

trainset_original <- trainset

table(trainset$diagnosis)
```

```
##
##    0    1
## 250 148
```

```
#Unbalanced dataset
#Logistic Regression
```

```
logmodel1 <- glm(diagnosis~.,data=trainset,family=binomial())
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(logmodel1)
```

```
##
## Call:
```

```
## glm(formula = diagnosis ~ ., family = binomial(), data = trainset)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -1.61412  -0.05381  -0.00730   0.00002   2.85617
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -22.70232    10.81320  -2.100  0.03577 *
## perimeter_mean    -0.16103     0.38640  -0.417  0.67686
## area_mean        -0.01161     0.02828  -0.411  0.68137
## concave.points_mean 103.87690    33.37978   3.112  0.00186 **
## perimeter_se       0.10166     1.50433   0.068  0.94612
## area_se           0.07246     0.10666   0.679  0.49692
## texture_worst      0.34112     0.08737   3.904 9.44e-05 ***
## perimeter_worst    -0.01669     0.26168  -0.064  0.94915
## area_worst         0.02913     0.01984   1.468  0.14198
## symmetry_worst     27.12355    11.41741   2.376  0.01752 *
## fractal_dimension_worst -17.70702    34.51639  -0.513  0.60795
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 525.311  on 397  degrees of freedom
## Residual deviance:  50.056  on 387  degrees of freedom
## AIC: 72.056
##
## Number of Fisher Scoring iterations: 10
```

#Remove insignificant variables

```
logmodel2 <- glm(diagnosis ~ perimeter_mean + concave.points_mean+ texture_worst + symmetry_worst,data=
summary(logmodel2)
```

```
##
## Call:
## glm(formula = diagnosis ~ perimeter_mean + concave.points_mean +
##      texture_worst + symmetry_worst, family = binomial(), data = trainset)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.0291  -0.1146  -0.0152   0.0073   3.3250
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -35.51120     5.72260  -6.205 5.45e-10 ***
## perimeter_mean     0.16185     0.03488   4.640 3.48e-06 ***
## concave.points_mean 77.42894    18.71614   4.137 3.52e-05 ***
## texture_worst      0.31844     0.06170   5.161 2.45e-07 ***
## symmetry_worst     27.19097     8.01890   3.391 0.000697 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
## Null deviance: 525.311 on 397 degrees of freedom
## Residual deviance: 74.137 on 393 degrees of freedom
## AIC: 84.137
##
## Number of Fisher Scoring iterations: 8

#Evaluate model with confusion matrix
logmodel2.test<-predict(logmodel2,newdata=testset,type='response')

#threshold = 0.9
logmodel2.predict.test<-ifelse(logmodel2.test>0.9,"1","0")
table(Testset.Actual = testset$diagnosis,logmodel2.predict.test)
```

```
##          logmodel2.predict.test
## Testset.Actual  0  1
##              0 106  1
##              1  7  57
```

```
mean(logmodel2.predict.test==testset$diagnosis)
```

```
## [1] 0.9532164
```

```
#Cart
set.seed(100)
Cart1 <- rpart(diagnosis ~.,data=trainset, method='class', control = rpart.control(minsplit = 2, cp=0.01))
print(Cart1)
```

```
## n= 398
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 398 148 0 (0.628140704 0.371859296)
##    2) perimeter_worst< 105.25 237 9 0 (0.962025316 0.037974684)
##      4) concave.points_mean< 0.07973 234 6 0 (0.974358974 0.025641026)
##        8) area_mean< 694.5 233 5 0 (0.978540773 0.021459227)
##          16) area_worst< 750.05 220 2 0 (0.990909091 0.009090909)
##            32) texture_worst< 33.35 205 0 0 (1.000000000 0.000000000) *
##            33) texture_worst>=33.35 15 2 0 (0.866666667 0.133333333)
##              66) concave.points_mean< 0.04887 14 1 0 (0.928571429 0.071428571)
##                132) texture_worst>=33.56 13 0 0 (1.000000000 0.000000000) *
##                133) texture_worst< 33.56 1 0 1 (0.000000000 1.000000000) *
##                  67) concave.points_mean>=0.04887 1 0 1 (0.000000000 1.000000000) *
##            17) area_worst>=750.05 13 3 0 (0.769230769 0.230769231)
##              34) symmetry_worst< 0.29 10 0 0 (1.000000000 0.000000000) *
##              35) symmetry_worst>=0.29 3 0 1 (0.000000000 1.000000000) *
##        9) area_mean>=694.5 1 0 1 (0.000000000 1.000000000) *
##      5) concave.points_mean>=0.07973 3 0 1 (0.000000000 1.000000000) *
##    3) perimeter_worst>=105.25 161 22 1 (0.136645963 0.863354037)
##      6) concave.points_mean< 0.048865 27 10 0 (0.629629630 0.370370370)
```

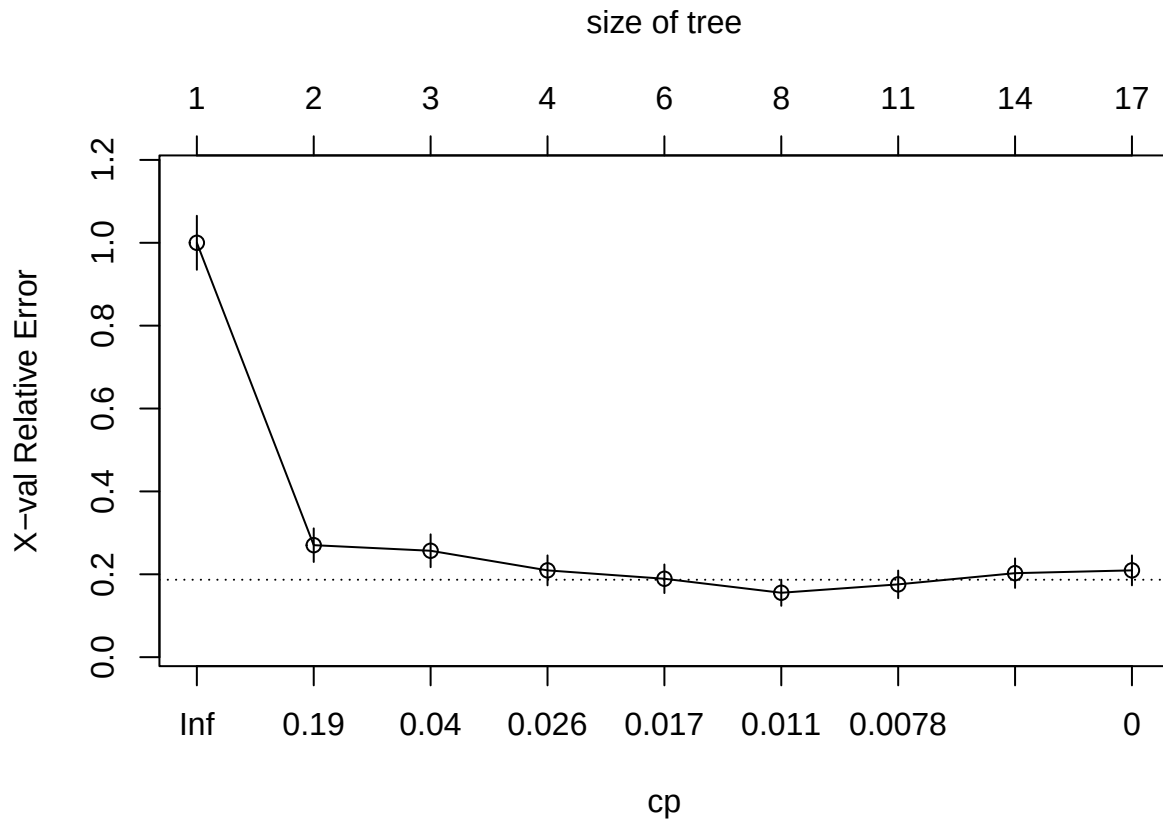


```
##      12) texture_worst< 26.03 14   1 0 (0.928571429 0.071428571)
##      24) symmetry_worst< 0.3262 13   0 0 (1.000000000 0.000000000) *
##      25) symmetry_worst>=0.3262 1   0 1 (0.000000000 1.000000000) *
##     13) texture_worst>=26.03 13   4 1 (0.307692308 0.692307692)
##     26) area_worst< 886.8 3   0 0 (1.000000000 0.000000000) *
##     27) area_worst>=886.8 10   1 1 (0.100000000 0.900000000)
##      54) perimeter_se>=3.261 1   0 0 (1.000000000 0.000000000) *
##      55) perimeter_se< 3.261 9   0 1 (0.000000000 1.000000000) *
##     7) concave.points_mean>=0.048865 134   5 1 (0.037313433 0.962686567)
##    14) texture_worst< 20.645 9   4 1 (0.444444444 0.555555556)
##     28) perimeter_mean< 102.25 4   0 0 (1.000000000 0.000000000) *
##     29) perimeter_mean>=102.25 5   0 1 (0.000000000 1.000000000) *
##    15) texture_worst>=20.645 125   1 1 (0.008000000 0.992000000)
##     30) area_worst< 724.15 1   0 0 (1.000000000 0.000000000) *
##     31) area_worst>=724.15 124   0 1 (0.000000000 1.000000000) *
```

```
#Viewing prune sequences, prune triggers, and 10-fold CV errors
printcp(Cart1)
```

```
##
## Classification tree:
## rpart(formula = diagnosis ~ ., data = trainset, method = "class",
##       control = rpart.control(minsplit = 2, cp = 0))
##
## Variables actually used in tree construction:
## [1] area_mean          area_worst          concave.points_mean
## [4] perimeter_mean     perimeter_se       perimeter_worst
## [7] symmetry_worst     texture_worst
##
## Root node error: 148/398 = 0.37186
##
## n= 398
##
##      CP nsplit rel error  xerror   xstd
## 1 0.7905405     0  1.000000 1.00000 0.065147
## 2 0.0472973     1  0.209459 0.27027 0.040529
## 3 0.0337838     2  0.162162 0.25676 0.039613
## 4 0.0202703     3  0.128378 0.20946 0.036125
## 5 0.0135135     5  0.087838 0.18919 0.034473
## 6 0.0090090     7  0.060811 0.15541 0.031454
## 7 0.0067568    10  0.033784 0.17568 0.033308
## 8 0.0045045    13  0.013514 0.20270 0.035586
## 9 0.0000000    16  0.000000 0.20946 0.036125
```

```
#Identifying optimal CP with plotted CP
plotcp(Cart1)
```



```
cp1 = sqrt(0.0135135*0.0090090)
```

```
#Pruning tree at cp = 0.01103373
```

```
cp1 = sqrt(0.0135135*0.0090090)
```

```
cp1
```

```
## [1] 0.01103373
```

```
Cart2<-prune(Cart1,cp=cp1)
```

```
printcp(Cart2)
```

```
##
```

```
## Classification tree:
```

```
## rpart(formula = diagnosis ~ ., data = trainset, method = "class",
```

```
## control = rpart.control(minsplit = 2, cp = 0))
```

```
##
```

```
## Variables actually used in tree construction:
```

```
## [1] area_worst concave.points_mean perimeter_mean
```

```
## [4] perimeter_worst texture_worst
```

```
##
```

```
## Root node error: 148/398 = 0.37186
```

```
##
```

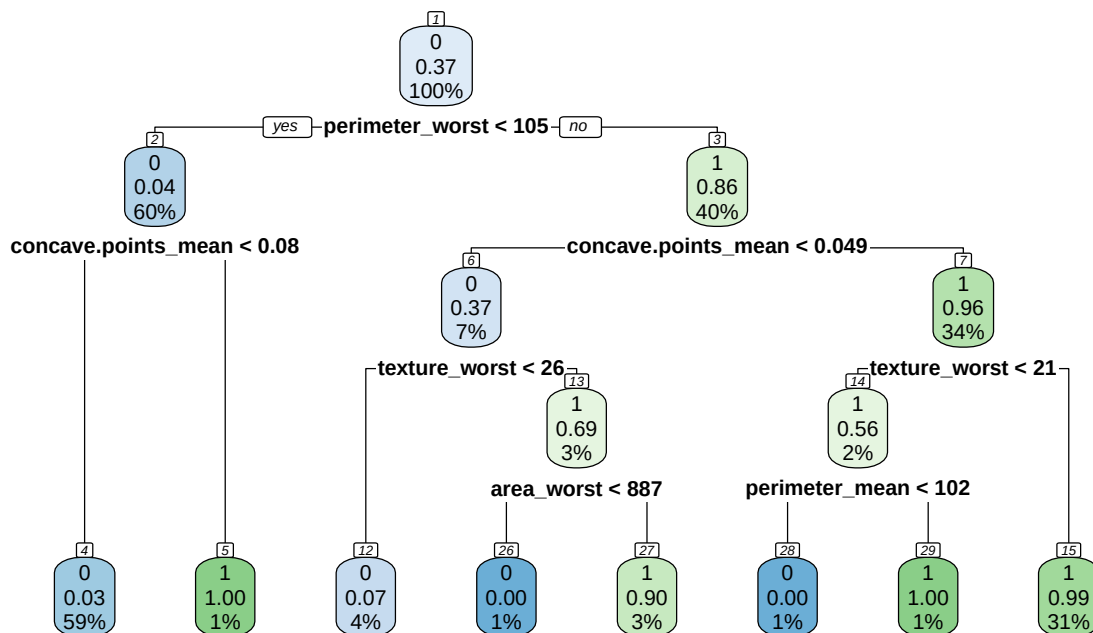
```
## n= 398
```

```
##
```

```
##          CP nsplit rel error  xerror    xstd
## 1 0.790541    0  1.000000 1.00000 0.065147
## 2 0.047297    1  0.209459 0.27027 0.040529
## 3 0.033784    2  0.162162 0.25676 0.039613
## 4 0.020270    3  0.128378 0.20946 0.036125
## 5 0.013514    5  0.087838 0.18919 0.034473
## 6 0.011034    7  0.060811 0.15541 0.031454
```

```
rpart.plot(Cart2, nn= T, main = "Pruned Tree with cp = 0.01103373")
```

Pruned Tree with cp = 0.01103373



```
Cart2$variable.importance
```

```
##          perimeter_worst          area_worst      concave.points_mean
##          136.316213          132.693970          121.950903
##          perimeter_mean          area_mean          area_se
##          120.065824          114.921260          101.456211
##          texture_worst fractal_dimension_worst      symmetry_worst
##          9.563418          4.995586          3.749432
##          perimeter_se
##          1.752013
```

```
#Evaluating predictive accuracy using confusion matrix
set.seed(100)
prob.test2<-predict(Cart2,newdata=testset,type='class')
table(prob.test2,testset$diagnosis)
```

```
##
## prob.test2  0  1
##           0 105  7
##           1  2 57
```

```
mean(prob.test2==testset$diagnosis)
```

```
## [1] 0.9473684
```

```
#RandomForest
set.seed(100)
rfmodel1 <- randomForest(trainset$diagnosis~ ., data = trainset, importance = T, ntree = 25, mtry = 6)
rfmodel1
```

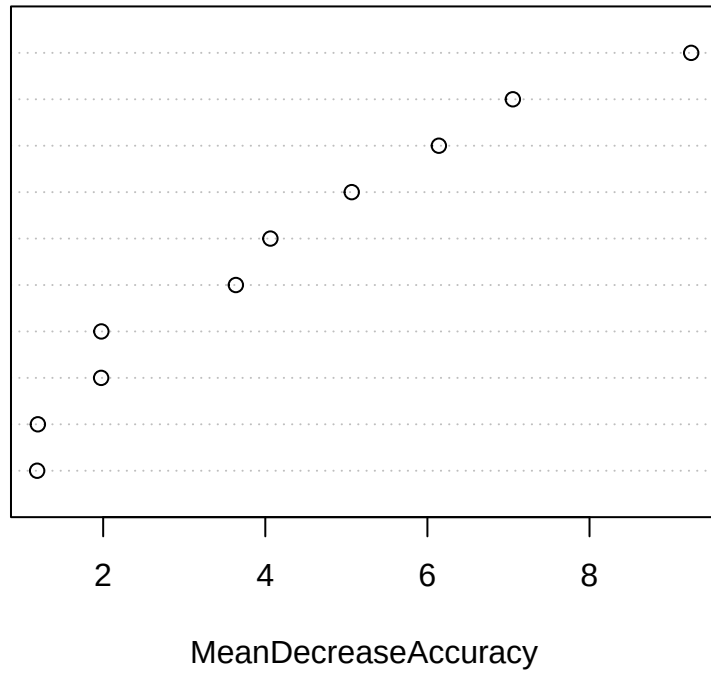
```
##
## Call:
## randomForest(formula = trainset$diagnosis ~ ., data = trainset, importance = T, ntree = 25, mtry = 6)
##           Type of random forest: classification
##           Number of trees: 25
## No. of variables tried at each split: 6
##
##           OOB estimate of  error rate: 4.77%
## Confusion matrix:
##      0   1 class.error
## 0 243   7 0.02800000
## 1  12 136 0.08108108
```

```
var.impt <- importance(rfmodel1)
```

```
varImpPlot(rfmodel1, type = 1)
```

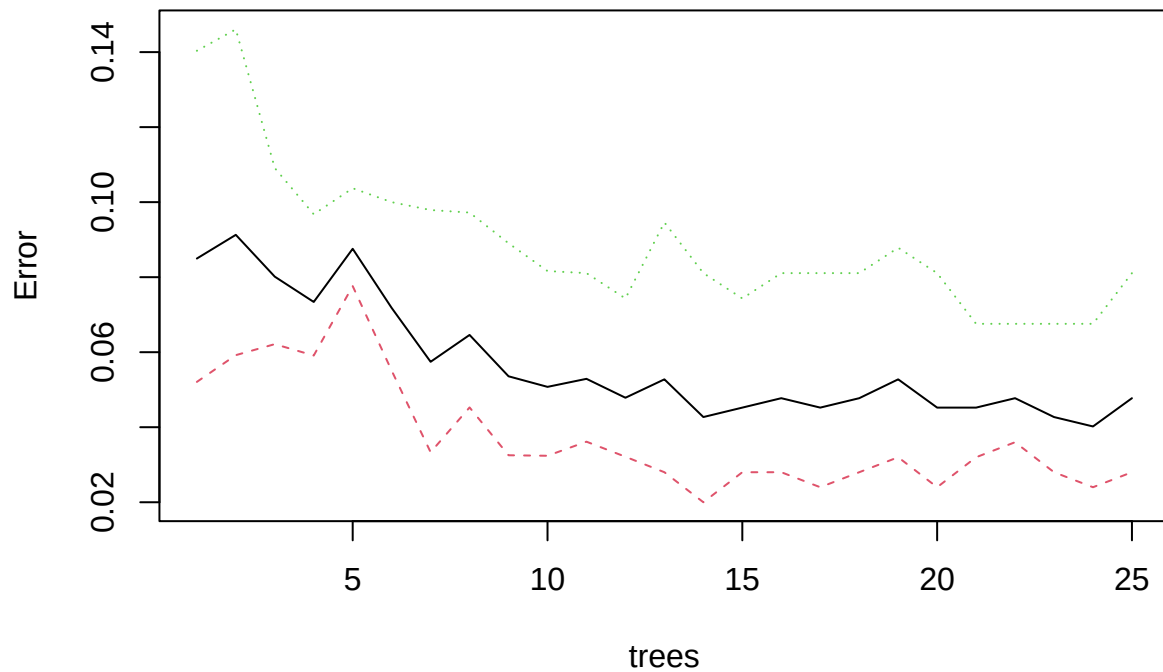
rfmodel1

concave.points_mean
area_worst
texture_worst
perimeter_worst
area_mean
symmetry_worst
perimeter_mean
perimeter_se
fractal_dimension_worst
area_se



```
plot(rfmodel1)
```

rfmodel1



```
names(testset) <- make.names(names(testset))
predictions <- predict(rfmodel1, newdata = testset)
table(testset.Actual = testset$diagnosis, Predict = predictions)
```

```
##          Predict
## testset.Actual  0   1
##                0 107  0
##                1   4  60
```

```
mean(testset$diagnosis == predictions)
```

```
## [1] 0.9766082
```

```
#Balanced dataset, downsampled
```

```
trainset <- downSample(trainset,trainset$diagnosis)
View(trainset)
table(trainset$diagnosis)
```

```
##
##    0   1
## 148 148
```

```
to_drop <- c("Class")
trainset <- select(trainset, -to_drop)
```

```
#Logistic Regression
```

```
logmodel1 <- glm(diagnosis~.,data=trainset,family=binomial())
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(logmodel1)
```

```
##
## Call:
## glm(formula = diagnosis ~ ., family = binomial(), data = trainset)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -1.88820  -0.01657  -0.00002   0.00002   2.24837
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -25.53060    22.04795  -1.158  0.24688
## perimeter_mean    -0.05209     0.73045  -0.071  0.94315
## area_mean        -0.02437     0.05311  -0.459  0.64632
## concave.points_mean 142.91208    65.34346   2.187  0.02874 *
## perimeter_se       1.93913     1.68502   1.151  0.24981
## area_se          -0.03128     0.11740  -0.266  0.78988
## texture_worst      0.59945     0.19671   3.047  0.00231 **
## perimeter_worst    -0.36833     0.45425  -0.811  0.41745
## area_worst         0.06364     0.03698   1.721  0.08525 .
## symmetry_worst     20.13331    15.48039   1.301  0.19341
## fractal_dimension_worst 28.49263    62.97731   0.452  0.65096
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 410.343  on 295  degrees of freedom
## Residual deviance:  27.166  on 285  degrees of freedom
## AIC: 49.166
##
## Number of Fisher Scoring iterations: 12
```

```
#Remove insignificant variables
```

```
logmodel2 <- glm(diagnosis ~ concave.points_mean+ texture_worst ,data=trainset,family=binomial())
summary(logmodel2)
```

```
##
## Call:
## glm(formula = diagnosis ~ concave.points_mean + texture_worst,
##      family = binomial(), data = trainset)
##
```

```
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.54066  -0.15546  -0.00350   0.06225   2.57238
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -17.09982     2.62519  -6.514 7.33e-11 ***
## concave.points_mean 135.09027    19.64723   6.876 6.16e-12 ***
## texture_worst      0.41098     0.07531   5.457 4.84e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 410.343  on 295  degrees of freedom
## Residual deviance:  85.353  on 293  degrees of freedom
## AIC: 91.353
##
## Number of Fisher Scoring iterations: 8
```

```
#Evaluate model with confusion matrix
logmodel2.test<-predict(logmodel2,newdata=testset,type='response')

#threshold = 0.9
logmodel2.predict.test<-ifelse(logmodel2.test>0.9,"1","0")
table(Testset.Actual = testset$diagnosis,logmodel2.predict.test)
```

```
##              logmodel2.predict.test
## Testset.Actual    0    1
##                0 104    3
##                1   7   57
```

```
mean(logmodel2.predict.test==testset$diagnosis)
```

```
## [1] 0.9415205
```

```
#Cart
set.seed(100)
Cart1 <- rpart(diagnosis ~.,data=trainset, method='class', control = rpart.control(minsplit = 2, cp=0))
print(Cart1)
```

```
## n= 296
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 296 148 0 (0.500000000 0.500000000)
##    2) perimeter_worst< 105.9 150 10 0 (0.933333333 0.066666667)
##      4) symmetry_worst< 0.3617 143 5 0 (0.965034965 0.034965035)
##        8) texture_worst< 33.35 137 2 0 (0.985401460 0.014598540)
##          16) area_mean< 694.5 136 1 0 (0.992647059 0.007352941)
```

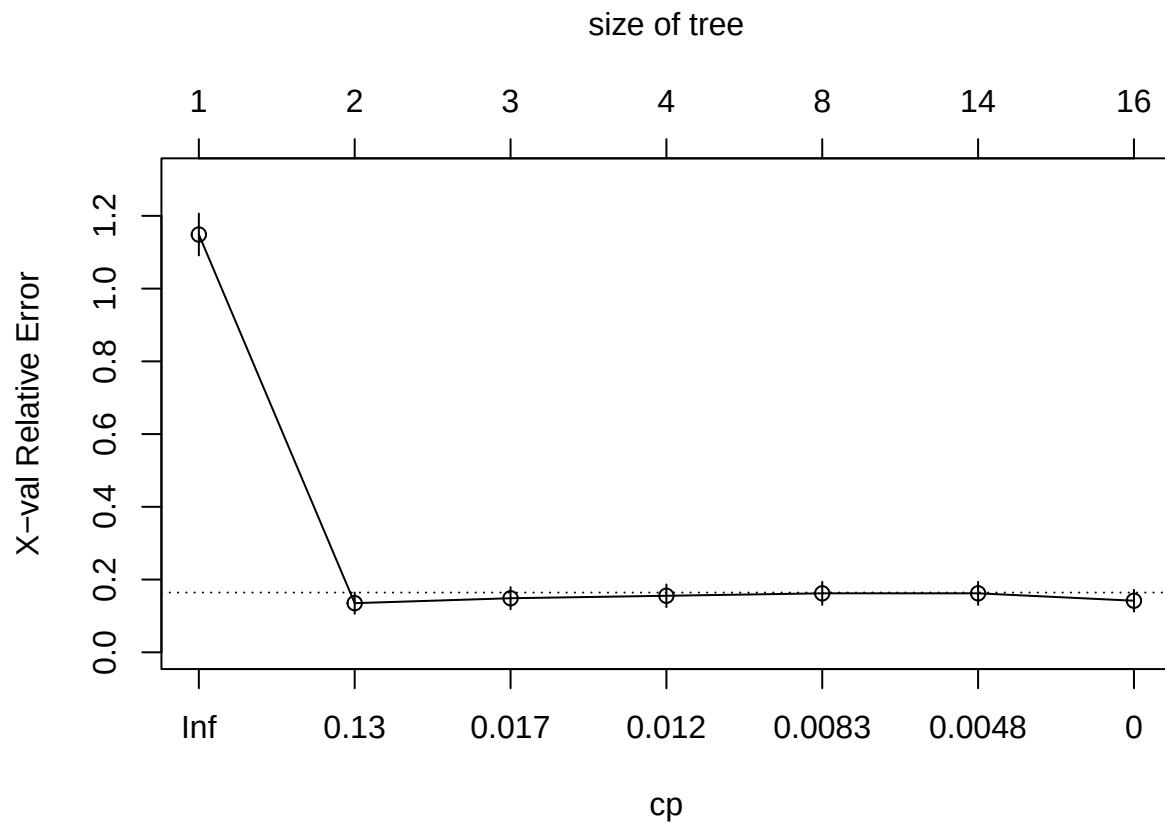


```
##          32) area_worst< 785 130    0 0 (1.000000000 0.000000000) *
##          33) area_worst>=785 6     1 0 (0.833333333 0.166666667)
##          66) perimeter_se>=1.6165 5    0 0 (1.000000000 0.000000000) *
##          67) perimeter_se< 1.6165 1    0 1 (0.000000000 1.000000000) *
##          17) area_mean>=694.5 1     0 1 (0.000000000 1.000000000) *
##          9) texture_worst>=33.35 6     3 0 (0.500000000 0.500000000)
##          18) perimeter_se< 1.447 3     0 0 (1.000000000 0.000000000) *
##          19) perimeter_se>=1.447 3     0 1 (0.000000000 1.000000000) *
##          5) symmetry_worst>=0.3617 7    2 1 (0.285714286 0.714285714)
##          10) perimeter_mean< 68.77 2    0 0 (1.000000000 0.000000000) *
##          11) perimeter_mean>=68.77 5    0 1 (0.000000000 1.000000000) *
##          3) perimeter_worst>=105.9 146   8 1 (0.054794521 0.945205479)
##          6) concave.points_mean< 0.04636 14    5 1 (0.357142857 0.642857143)
##          12) area_worst< 893.65 3     0 0 (1.000000000 0.000000000) *
##          13) area_worst>=893.65 11     2 1 (0.181818182 0.818181818)
##          26) concave.points_mean>=0.045085 1    0 0 (1.000000000 0.000000000) *
##          27) concave.points_mean< 0.045085 10    1 1 (0.100000000 0.900000000)
##          54) perimeter_se>=3.261 1     0 0 (1.000000000 0.000000000) *
##          55) perimeter_se< 3.261 9     0 1 (0.000000000 1.000000000) *
##          7) concave.points_mean>=0.04636 132    3 1 (0.022727273 0.977272727)
##          14) area_worst< 724.15 1     0 0 (1.000000000 0.000000000) *
##          15) area_worst>=724.15 131    2 1 (0.015267176 0.984732824)
##          30) texture_worst< 20.645 7     2 1 (0.285714286 0.714285714)
##          60) perimeter_mean< 102.25 2    0 0 (1.000000000 0.000000000) *
##          61) perimeter_mean>=102.25 5    0 1 (0.000000000 1.000000000) *
##          31) texture_worst>=20.645 124    0 1 (0.000000000 1.000000000) *
```

```
#Viewing prune sequences, prune triggers, and 10-fold CV errors
printcp(Cart1)
```

```
##
## Classification tree:
## rpart(formula = diagnosis ~ ., data = trainset, method = "class",
##       control = rpart.control(minsplit = 2, cp = 0))
##
## Variables actually used in tree construction:
## [1] area_mean          area_worst          concave.points_mean
## [4] perimeter_mean     perimeter_se       perimeter_worst
## [7] symmetry_worst     texture_worst
##
## Root node error: 148/296 = 0.5
##
## n= 296
##
##      CP nsplit rel error  xerror    xstd
## 1 0.8783784      0 1.0000000 1.14865 0.057478
## 2 0.0202703      1 0.1216216 0.13514 0.029178
## 3 0.0135135      2 0.1013514 0.14865 0.030492
## 4 0.0101351      3 0.0878378 0.15541 0.031120
## 5 0.0067568      7 0.0472973 0.16216 0.031731
## 6 0.0033784     13 0.0067568 0.16216 0.031731
## 7 0.0000000     15 0.0000000 0.14189 0.029845
```

```
#Identifying optimal CP with plotted CP
plotcp(Cart1)
```



```
cp1 = sqrt(0.8783784*0.0202703)
```

```
#Pruning tree at cp = 0.1334354
cp1 = sqrt(0.8783784*0.0202703)
cp1
```

```
## [1] 0.1334354
```

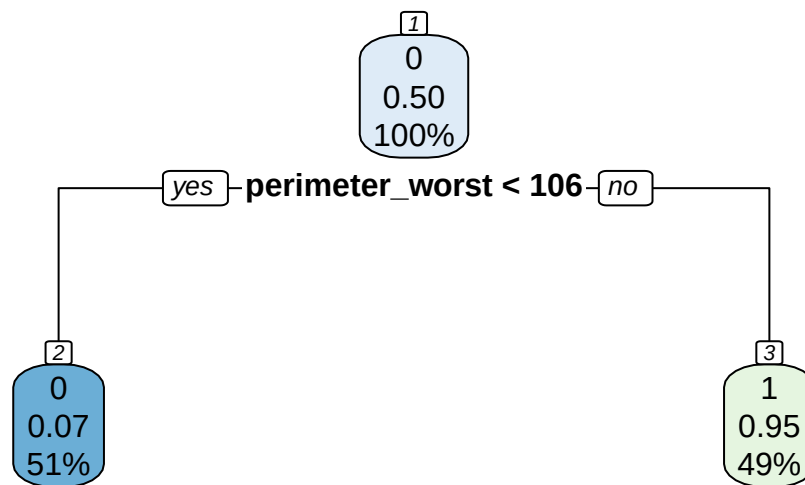
```
Cart2<-prune(Cart1,cp=cp1)
printcp(Cart2)
```

```
##
## Classification tree:
## rpart(formula = diagnosis ~ ., data = trainset, method = "class",
##       control = rpart.control(minsplit = 2, cp = 0))
##
## Variables actually used in tree construction:
## [1] perimeter_worst
##
## Root node error: 148/296 = 0.5
##
```

```
## n= 296
##
##      CP nsplit rel error  xerror   xstd
## 1 0.87838      0  1.00000 1.14865 0.057478
## 2 0.13344      1  0.12162 0.13514 0.029178
```

```
rpart.plot(Cart2, nn= T, main = "Pruned Tree with cp = 0.1334354")
```

Pruned Tree with cp = 0.1334354



```
Cart2$variable.importance
```

```
##      perimeter_worst      area_worst      perimeter_mean      area_mean
##      114.21005         107.95196         99.34709         94.65353
## concave.points_mean      area_se
##      91.52449         89.17771
```

```
#Evaluating predictive accuracy using confusion matrix
set.seed(100)
prob.test2<-predict(Cart1,newdata=testset,type='class')
table(prob.test2,testset$diagnosis)
```

```
##
## prob.test2  0  1
##           0 97  5
##           1 10 59
```

```
mean(prob.test2==testset$diagnosis)
```

```
## [1] 0.9122807
```

```
#RandomForest
```

```
set.seed(100)
```

```
rfmodel1 <- randomForest(trainset$diagnosis~ ., data = trainset, importance = T, ntree = 25, mtry = 6)  
rfmodel1
```

```
##
```

```
## Call:
```

```
## randomForest(formula = trainset$diagnosis ~ ., data = trainset, importance = T, ntree = 25, mt.
```

```
##           Type of random forest: classification
```

```
##           Number of trees: 25
```

```
## No. of variables tried at each split: 6
```

```
##
```

```
##           OOB estimate of error rate: 6.08%
```

```
## Confusion matrix:
```

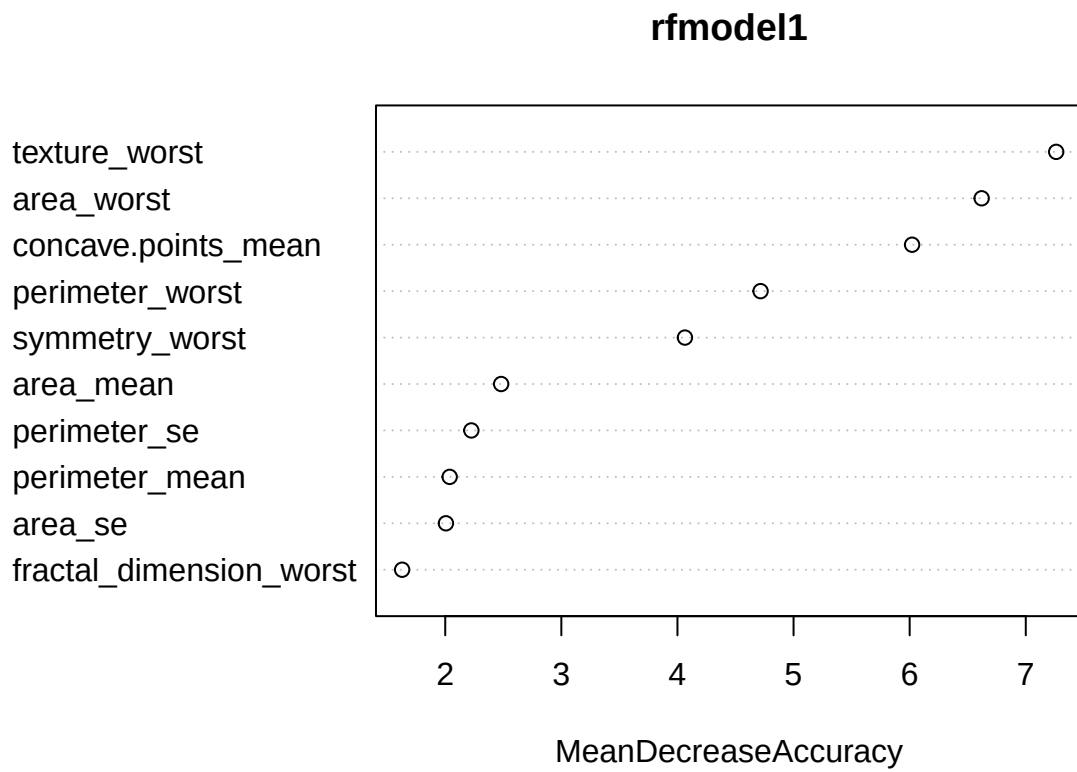
```
##      0   1 class.error
```

```
## 0 139   9 0.06081081
```

```
## 1   9 139 0.06081081
```

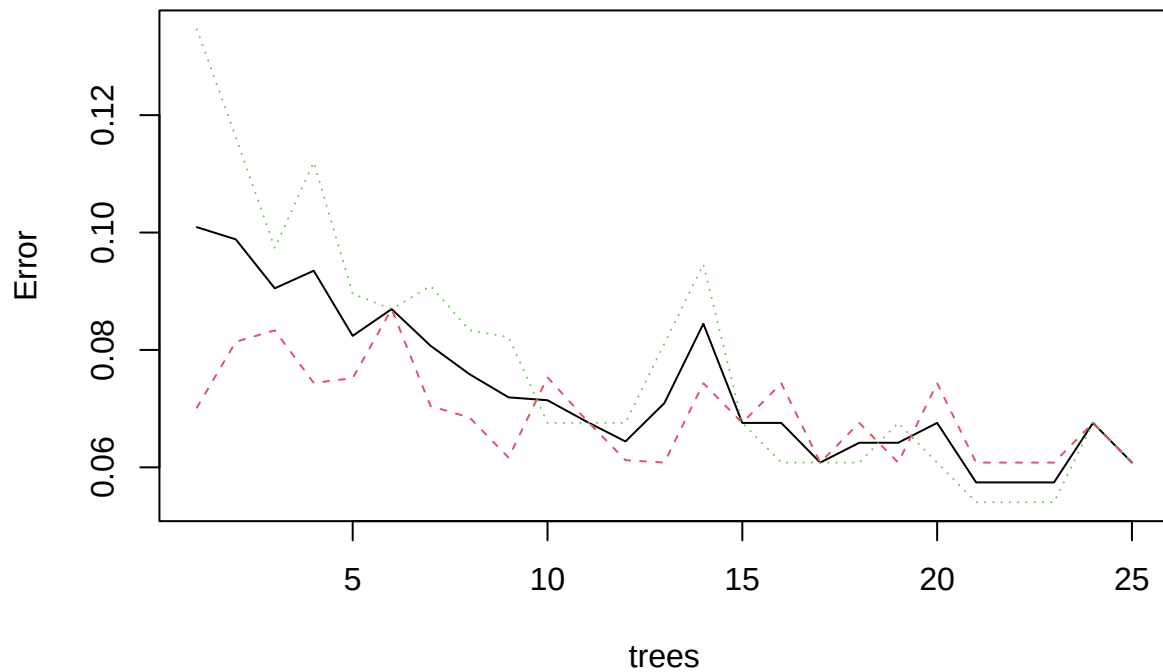
```
var.impt <- importance(rfmodel1)
```

```
varImpPlot(rfmodel1, type = 1)
```



```
plot(rfmodel1)
```

rfmodel1



```
names(testset) <- make.names(names(testset))
predictions <- predict(rfmodel1, newdata = testset)
table(testset.Actual = testset$diagnosis, Predict = predictions)
```

```
##           Predict
## testset.Actual  0   1
##                0 100   7
##                1   2  62
```

```
mean(testset$diagnosis == predictions)
```

```
## [1] 0.9473684
```

```
#Balanced dataset, upsampled
```

```
trainset <- upSample(trainset_original, trainset_original$diagnosis)
View(trainset)
```

```
to_drop <- c("Class")
trainset <- select(trainset, -to_drop)
```

```
table(trainset$diagnosis)
```

```
##
##    0    1
## 250 250
```

#Logistic Regression

```
logmodel1 <- glm(diagnosis~.,data=trainset,family=binomial())
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(logmodel1)
```

```
##
## Call:
## glm(formula = diagnosis ~ ., family = binomial(), data = trainset)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -1.7766  -0.0425  -0.0002   0.0005   3.6514
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -20.916207   10.321163  -2.027  0.042710 *
## perimeter_mean    -0.253798    0.392923  -0.646  0.518328
## area_mean        -0.006302    0.028934  -0.218  0.827581
## concave.points_mean 118.104227  30.746486   3.841  0.000122 ***
## perimeter_se      -0.833463    1.581364  -0.527  0.598157
## area_se           0.104495    0.111775   0.935  0.349854
## texture_worst      0.336837    0.073491   4.583  4.58e-06 ***
## perimeter_worst     0.068730    0.262256   0.262  0.793266
## area_worst         0.024437    0.019243   1.270  0.204108
## symmetry_worst     24.755097    9.727508   2.545  0.010932 *
## fractal_dimension_worst -15.827776  30.760520  -0.515  0.606869
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 693.147  on 499  degrees of freedom
## Residual deviance:  65.766  on 489  degrees of freedom
## AIC: 87.766
##
## Number of Fisher Scoring iterations: 10
```

#Remove insignificant variables

```
logmodel2 <- glm(diagnosis ~ perimeter_mean + concave.points_mean+ texture_worst + symmetry_worst ,data=trainset)
summary(logmodel2)
```

```
##
## Call:
## glm(formula = diagnosis ~ perimeter_mean + concave.points_mean +
##      texture_worst + symmetry_worst, family = binomial(), data = trainset)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.3082  -0.0819  -0.0005   0.0170   3.2290
```

```
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -36.72057    5.24054  -7.007 2.43e-12 ***
## perimeter_mean    0.17252    0.03068   5.623 1.88e-08 ***
## concave.points_mean 76.25669   16.85144   4.525 6.03e-06 ***
## texture_worst     0.32912    0.05647   5.828 5.61e-09 ***
## symmetry_worst    28.91864    7.41901   3.898 9.70e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 693.15  on 499  degrees of freedom
## Residual deviance:  94.88  on 495  degrees of freedom
## AIC: 104.88
##
## Number of Fisher Scoring iterations: 9
```

```
#Evaluate model with confusion matrix
logmodel2.test<-predict(logmodel2,newdata=testset,type='response')

#threshold = 0.9
logmodel2.predict.test<-ifelse(logmodel2.test>0.9,"1","0")
table(Testset.Actual = testset$diagnosis,logmodel2.predict.test)
```

```
##              logmodel2.predict.test
## Testset.Actual    0    1
##                0 106    1
##                1   6   58
```

```
mean(logmodel2.predict.test==testset$diagnosis)
```

```
## [1] 0.9590643
```

```
#Cart
set.seed(100)
Cart1 <- rpart(diagnosis ~.,data=trainset, method='class', control = rpart.control(minsplit = 2, cp=0))

print(Cart1)
```

```
## n= 500
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 500 250 0 (0.5000000000 0.5000000000)
## 2) perimeter_worst< 105.25 243 15 0 (0.938271605 0.061728395)
## 4) concave.points_mean< 0.07973 237 9 0 (0.962025316 0.037974684)
## 8) texture_worst< 33.35 218 4 0 (0.981651376 0.018348624)
## 16) area_mean< 694.5 217 3 0 (0.986175115 0.013824885)
## 32) perimeter_worst< 102.4 207 0 0 (1.0000000000 0.0000000000) *
```



```

##          33) perimeter_worst>=102.4 10    3 0 (0.700000000 0.300000000)
##          66) texture_worst< 26.675 7     0 0 (1.000000000 0.000000000) *
##          67) texture_worst>=26.675 3     0 1 (0.000000000 1.000000000) *
##          17) area_mean>=694.5 1      0 1 (0.000000000 1.000000000) *
##          9) texture_worst>=33.35 19     5 0 (0.736842105 0.263157895)
##          18) concave.points_mean< 0.04485 16    2 0 (0.875000000 0.125000000)
##          36) texture_worst>=33.56 14     0 0 (1.000000000 0.000000000) *
##          37) texture_worst< 33.56 2      0 1 (0.000000000 1.000000000) *
##          19) concave.points_mean>=0.04485 3     0 1 (0.000000000 1.000000000) *
##          5) concave.points_mean>=0.07973 6     0 1 (0.000000000 1.000000000) *
##          3) perimeter_worst>=105.25 257  22 1 (0.085603113 0.914396887)
##          6) concave.points_mean< 0.048865 35   17 1 (0.485714286 0.514285714)
##          12) area_worst< 960.5 16      2 0 (0.875000000 0.125000000)
##          24) texture_worst< 33.5 14      0 0 (1.000000000 0.000000000) *
##          25) texture_worst>=33.5 2       0 1 (0.000000000 1.000000000) *
##          13) area_worst>=960.5 19      3 1 (0.157894737 0.842105263)
##          26) fractal_dimension_worst< 0.06492 3    0 0 (1.000000000 0.000000000) *
##          27) fractal_dimension_worst>=0.06492 16    0 1 (0.000000000 1.000000000) *
##          7) concave.points_mean>=0.048865 222    5 1 (0.022522523 0.977477477)
##          14) texture_worst< 20.645 11     4 1 (0.363636364 0.636363636)
##          28) perimeter_mean< 102.25 4      0 0 (1.000000000 0.000000000) *
##          29) perimeter_mean>=102.25 7      0 1 (0.000000000 1.000000000) *
##          15) texture_worst>=20.645 211    1 1 (0.004739336 0.995260664)
##          30) area_worst< 724.15 1       0 0 (1.000000000 0.000000000) *
##          31) area_worst>=724.15 210     0 1 (0.000000000 1.000000000) *

```

```

#Viewing prune sequences, prune triggers, and 10-fold CV errors
printcp(Cart1)

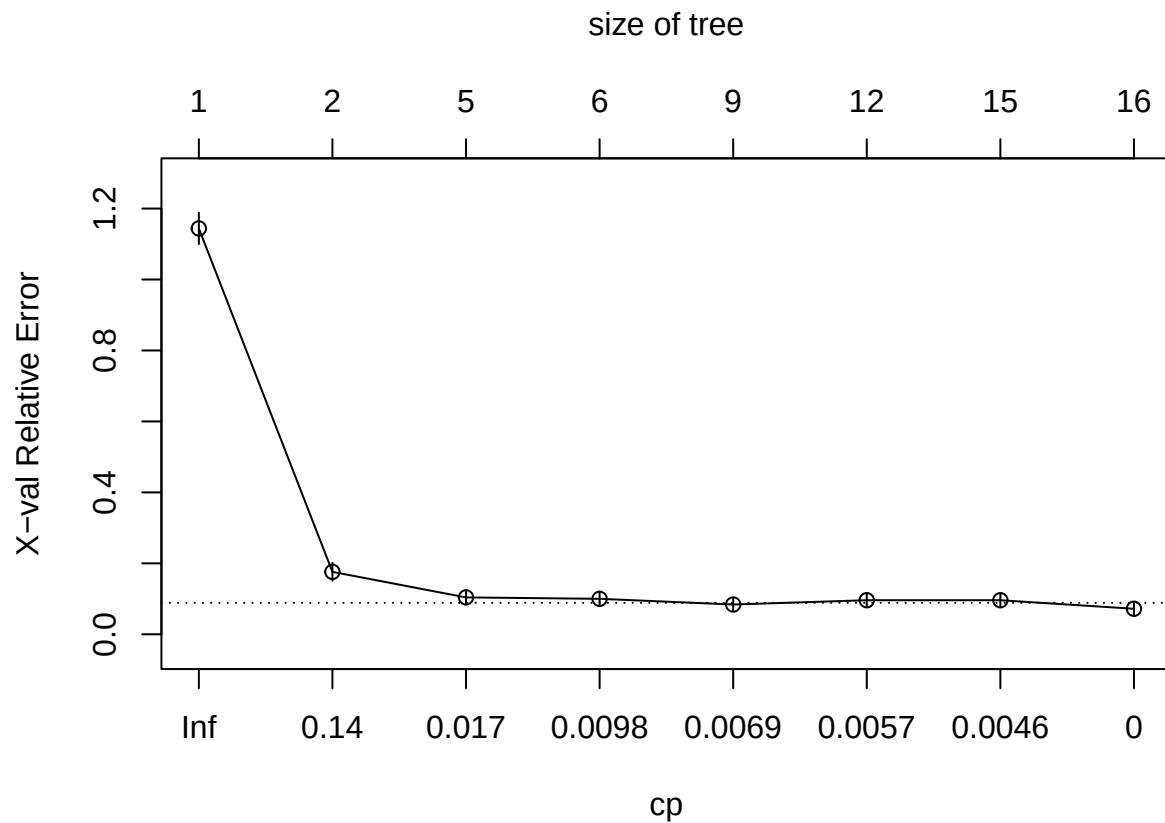
```

```

##
## Classification tree:
## rpart(formula = diagnosis ~ ., data = trainset, method = "class",
##       control = rpart.control(minsplit = 2, cp = 0))
##
## Variables actually used in tree construction:
## [1] area_mean          area_worst          concave.points_mean
## [4] fractal_dimension_worst perimeter_mean      perimeter_worst
## [7] texture_worst
##
## Root node error: 250/500 = 0.5
##
## n= 500
##
##      CP nsplit rel error xerror   xstd
## 1 0.8520000      0    1.000  1.144 0.044255
## 2 0.0240000      1    0.148  0.176 0.025339
## 3 0.0120000      4    0.076  0.104 0.019859
## 4 0.0080000      5    0.064  0.100 0.019494
## 5 0.0060000      8    0.040  0.084 0.017941
## 6 0.0053333     11    0.020  0.096 0.019120
## 7 0.0040000     14    0.004  0.096 0.019120
## 8 0.0000000     15    0.000  0.072 0.016662

```

```
#Identifying optimal CP with plotted CP
plotcp(Cart1)
```



```
cp1 = sqrt(0.006*0.008)

#Pruning tree at cp = 0.0069
cp1 = sqrt(0.006*0.008)
cp1
```

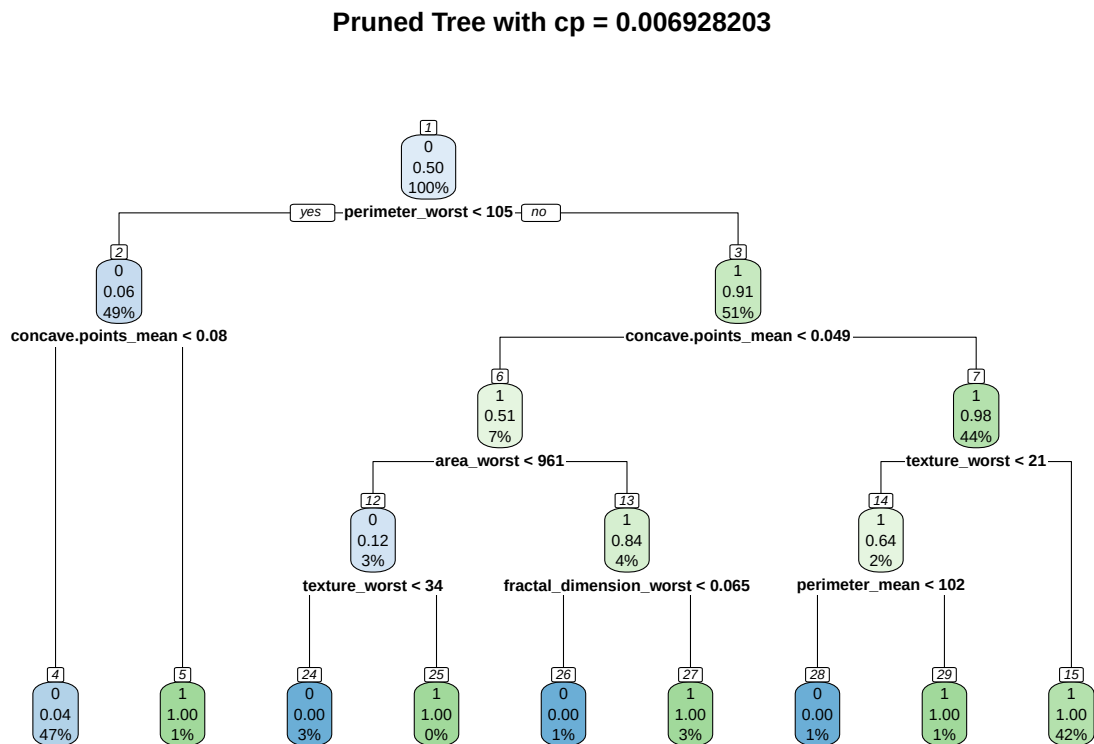
```
## [1] 0.006928203
```

```
Cart2<-prune(Cart1,cp=cp1)
printcp(Cart2)
```

```
##
## Classification tree:
## rpart(formula = diagnosis ~ ., data = trainset, method = "class",
##       control = rpart.control(minsplit = 2, cp = 0))
##
## Variables actually used in tree construction:
## [1] area_worst          concave.points_mean    fractal_dimension_worst
## [4] perimeter_mean      perimeter_worst        texture_worst
##
## Root node error: 250/500 = 0.5
```

```
##
## n= 500
##
##      CP nsplit rel error xerror   xstd
## 1 0.8520000    0    1.000  1.144 0.044255
## 2 0.0240000    1    0.148  0.176 0.025339
## 3 0.0120000    4    0.076  0.104 0.019859
## 4 0.0080000    5    0.064  0.100 0.019494
## 5 0.0069282    8    0.040  0.084 0.017941
```

```
rpart.plot(Cart2, nn= T, main = "Pruned Tree with cp = 0.006928203")
```



```
Cart2$variable.importance
```

```
##      perimeter_worst      area_worst      perimeter_mean
##      194.525745      187.420972      173.040206
##      concave.points_mean      area_mean      area_se
##      168.848930      165.788272      150.702412
##      texture_worst      symmetry_worst      fractal_dimension_worst
##      13.895127      12.394831      12.273760
##      perimeter_se
##      7.642635
```

```
#Evaluating predictive accuracy using confusion matrix)
set.seed(100)
prob.test2<-predict(Cart1,newdata=testset,type='class')
table(prob.test2,testset$diagnosis)
```

```
##
## prob.test2    0    1
##           0 102    6
##           1   5  58
```

```
mean(prob.test2==testset$diagnosis)
```

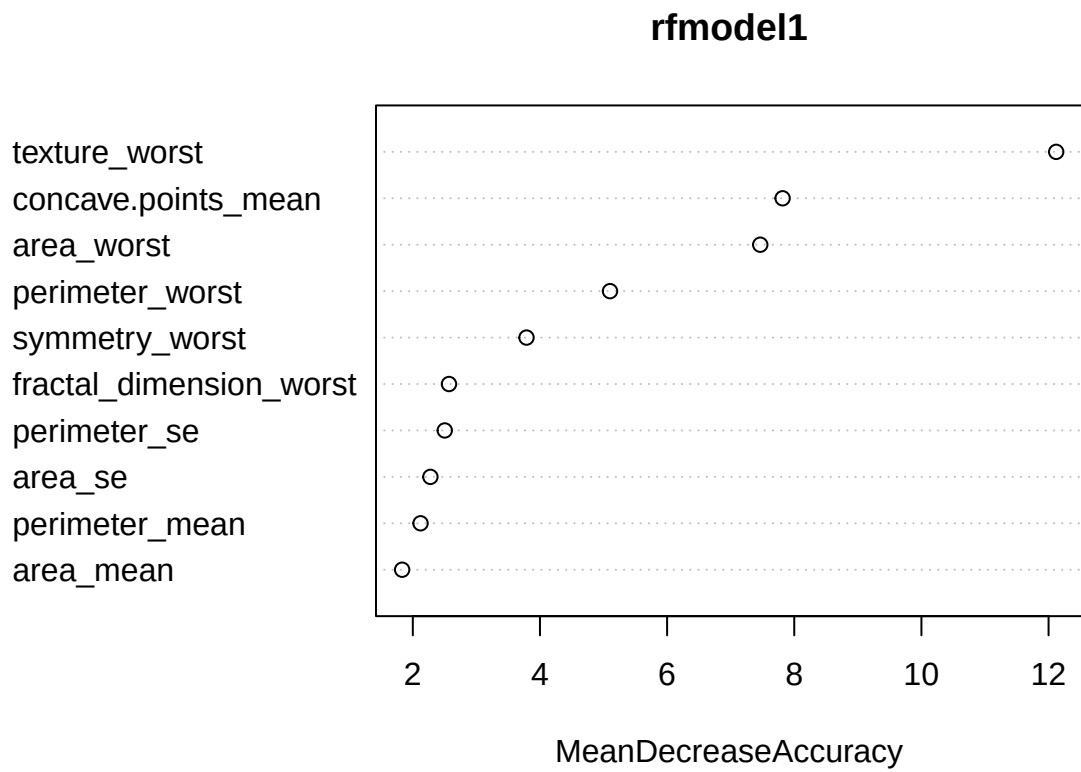
```
## [1] 0.9356725
```

```
#RandomForest
set.seed(100)
rfmodel1 <-randomForest(trainset$diagnosis~ ., data = trainset, importance = T, ntree = 25,mtry =6)
rfmodel1
```

```
##
## Call:
## randomForest(formula = trainset$diagnosis ~ ., data = trainset,      importance = T, ntree = 25, mt
##           Type of random forest: classification
##           Number of trees: 25
## No. of variables tried at each split: 6
##
##           OOB estimate of  error rate: 2.6%
## Confusion matrix:
##      0    1 class.error
## 0 241    9      0.036
## 1   4 246      0.016
```

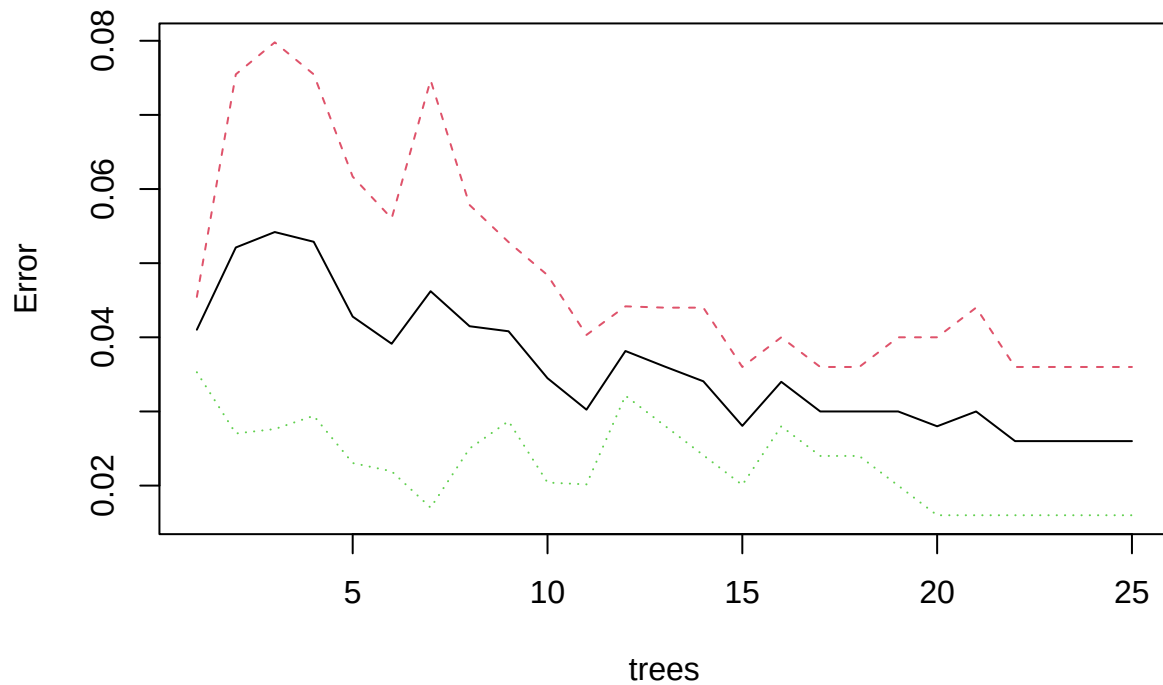
```
var.impt <- importance(rfmodel1)
```

```
varImpPlot(rfmodel1, type = 1)
```



```
plot(rfmodel1)
```

rfmodel1



```
names(testset) <- make.names(names(testset))
predictions <- predict(rfmodel1, newdata = testset)
table(testset.Actual = testset$diagnosis, Predict = predictions)
```

```
##          Predict
## testset.Actual  0   1
##              0 107   0
##              1   2  62
```

```
mean(testset$diagnosis == predictions)
```

```
## [1] 0.9883041
```